



A Mini Project Report on

AI POWERED PERSONALISED LEARNING PATH GENERATOR

Submitted in partial fulfilment of the requirements for the award of the degree of

Bachelor of Engineering
in
Computer Science and Engineering
Submitted by

SHREYASH BADIGER
M. VINAY KUMAR
GUNAVARDHAN

(1608-23-733-141)
(1608-23-733-195)
(1608-23-733-179)

Under the guidance of
Mrs. Bhagyalaxmi K
Assistant Professor

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Matrusri Engineering College

(Affiliated to Osmania University, Approved by AICTE)

Saidabad, Hyderabad - 500059

(2025-2026)



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Matrusri Engineering College

(Affiliated to Osmania University, Approved by AICTE)

Saidabad, Hyderabad – 500059

CERTIFICATE

This is to certify that the Mini Project report entitled "AI Powered Personalised Learning Path Generator" is being submitted by Shreyash Badiger (1608-23-733-141), M. Vinay Kumar (1608-23-733-195), and Gunavardhan (1608-23-733-179) in partial fulfilment of the requirements for the award of the degree of Bachelor of Engineering in "Computer Science and Engineering," Osmania University, Hyderabad, during the academic year 2025-2026. This is a record of bonafide work carried out by them under my guidance. The results presented in this project have been verified and are found to be satisfactory.

Project Coordinator

Mrs. M. Priyanka
Assistant Professor
Dept. of CSE

Project Guide

Mrs. Bhagyalaxmi K
Assistant Professor
Dept. of CSE

H.O.D

Dr. T. Raghunadha Reddy
Professor & Head
Dept. of CSE

External Examiner(s): _____



Department of Computer Science and Engineering Matrusri Engineering College

Accredited by NBA & NAAC

(Affiliated to Osmania University, Approved by AICTE) Saidabad, Hyderabad-500059
(2024-2025)

DECLARATION

We, Shreyash Badiger bearing Ht.No.1608-23-733-141, M. Vinay Kumar bearing Ht.No.1608-23-733-195, and Gunavardhan bearing Ht.No.1608-23-733-179, hereby certify that the mini project report entitled "AI Powered Personalised Learning Path Generator" is submitted in partial fulfilment of the requirements for the award of the degree of Bachelor of Engineering in Computer Science and Engineering.

This is a record of original work carried out by us under the guidance of Mrs. Bhagyalaxmi K, Assistant Professor, Department of Computer Science and Engineering, Matrusri Engineering College, Saidabad. The results embodied in this report have not been reproduced or copied from any other source, and have not been submitted to any other university or institute for the award of any degree or diploma.

Shreyash Badiger (1608-23-733-141) _____

M. Vinay Kumar (1608-23-733-195) _____

Gunavardhan (1608-23-733-179) _____

ACKNOWLEDGEMENT

It is our privilege and pleasure to express our profound sense of respect, gratitude, and indebtedness to our guide Mrs. Bhagyalaxmi K, Assistant Professor, Department of Computer Science and Engineering, Matrusri Engineering College, for her valuable inspiration, guidance, cogent discussion, constructive criticisms, and consistent encouragement throughout this dissertation work.

We express our sincere thanks to Project Coordinator Mrs. M. Priyanka, Assistant Professor, Department of Computer Science and Engineering, Matrusri Engineering College, for her valuable suggestions and constant help in completing the work.

We express our sincere gratitude to Dr. T. Raghunadha Reddy, Professor & Head, Department of Computer Science and Engineering, Matrusri Engineering College, for his precious suggestions, motivation, and co-operation.

We express our sincere thanks to Dr. D. Hanumantha Rao, Principal, Matrusri Engineering College, Saidabad, Hyderabad, for his encouragement and constant support.

We extend our sincere thanks to all the teaching and non-teaching staff of the Computer Science and Engineering Department for their support, cooperation, and guidance.

Last but not least, we wish to acknowledge our parents, family members, and friends for giving us moral strength, financial support, and encouraging us to complete this dissertation work successfully.

ABSTRACT

Existing e-learning platforms such as Coursera, Udemy, and YouTube adopt a one-size-fits-all model that fails to consider individual skill levels, learning pace, or career goals, leaving students overwhelmed and without a structured direction. The AI Powered Personalised Learning Path Generator (PathAI) is an intelligent, web-based platform that addresses this critical gap in online education. The system collects the student's academic profile, baseline experience, and primary learning goal spanning Job Placement, Skill Building, Exam Preparation, and Competitive Coding, and then conducts a short, MCQ-based skill assessment quiz to identify the appropriate technical starting point.

An advanced AI engine, integrated with the Groq API (using the high-performance Llama 3.3 70B Versatile model), processes these inputs to generate a highly personalized, step-by-step learning path. This roadmap is structured into modular chapters, each comprising a progressive set of topics, specific learning objectives, and curated online resources. To provide rich learning content, PathAI integrates with the YouTube Data API v3 to automatically fetch the most relevant, highly-rated tutorial video for each specific topic.

Additionally, learners can upload text documents (PDFs) related to their coursework to a dedicated Supabase Storage bucket, allowing the system to link custom resources directly to their learning path. An interactive, animated roadmap dashboard designed using React.js and Tailwind CSS lets students track completed topics in real-time. To reinforce concept retention, the platform generates checkpoint assessment quizzes at the end of each module using the same LLM. Built using a modern technical stack comprising a FastAPI (Python) backend, a React.js (Vite) frontend, and a Supabase PostgreSQL database, the system functions as an autonomous personal learning mentor that adapts to each student's profile and guides them toward their defined goal, making self-directed learning more structured, personalized, and outcome-driven.

TABLE OF CONTENTS

Chapter / Section	Title	Page No.
—	Abstract	v
—	Table of Contents	vi
—	List of Figures	vii
—	List of Tables	viii
Chapter 1	Introduction	1
1.1	Introduction to Project	1
1.2	Project Category	2
1.3	Objectives	2
1.4	Scope of the Problem	3
1.5	Identification of Need	3
1.6	Existing System	4
1.7	Limitations of the Existing System	4
1.8	Proposed System	5
1.9	Unique Features of the System	5
Chapter 2	Requirement Analysis & SRS	6
2.1	Feasibility Study	6
2.2	Software Requirement Specification (SRS)	7
2.3	Validation	10
2.4	Expected Hurdles	10
2.5	SDLC Model	11
Chapter 3	System Design	12
3.1	Design Approach	12
3.2	System Architecture	12
3.3	UML Diagrams	13
3.4	Interface Relationship & Dependencies	20
3.5	Database Design	21

Chapter / Section	Title	Page No.
3.6	User Interface Design	24
3.7	REST API Endpoints	25
Chapter 4	Implementation, Testing & Maintenance	26
4.1	Tools and Technologies Used	26
4.2	Coding Standards	27
4.3	Testing Techniques	27
4.4	Executable Code Listings	28
Chapter 5	Results and Discussions	36
5.1	User Interface Representation	36
5.2	System Screenshots	37
5.3	Detailed Test Cases	38
Chapter 6	Conclusion and Future Scope	43
6.1	Conclusion	43
6.2	Future Scope	43
—	References	44

LIST OF TABLES

Table No.	Table Title	Page No.
Table 3.1	onboarding_data Table — Data Dictionary	21
Table 3.2	learning_paths Table — Data Dictionary	22
Table 3.3	path_topics Table — Data Dictionary	22
Table 3.4	quiz_results Table — Data Dictionary	23
Table 3.5	checkpoint_results Table — Data Dictionary	24
Table 3.6	REST API Endpoints	25
Table 5.1	User Authentication Test Cases	38
Table 5.2	Onboarding & Quiz Generation Test Cases	39
Table 5.3	Roadmap & YouTube Integration Test Cases	40
Table 5.4	Checkpoint & PDF Upload Test Cases	41

LIST OF FIGURES

Figure No.	Figure Title	Page No.
Figure 3.1	Multi-layered System Architecture Diagram	12
Figure 3.2	Component Diagram of PathAI Platform	13
Figure 3.3	Object Diagram — Runtime Instances	14
Figure 3.4	Class Diagram — Database Models & Relationships	15
Figure 3.5	Deployment Diagram — Hardware Nodes & Interfaces	16
Figure 3.6	Use Case Diagram — Student Learner Role	17
Figure 3.7	State Transition Diagram — Path Lifecycle	18
Figure 3.8	Activity Diagram — Learning Path Workflow	19
Figure 3.9	Sequence Diagram — Path Generation Interaction	20
Figure 3.10	Interface Relationship & Dependencies Chart	20
Figure 5.1	Authentication Page Screenshot	37
Figure 5.2	Onboarding Form Screenshot	37
Figure 5.3	Initial Assessment Quiz Screenshot	38
Figure 5.4	Roadmap Dashboard Screenshot	38
Figure 5.5	Topic Card Details Screenshot	39
Figure 5.6	Checkpoint Quiz Modal Screenshot	39
Figure 5.7	PDF Sidebar Panel Screenshot	40

Chapter 1: Introduction

1.1 Introduction to Project

In the contemporary digital era, the availability of educational resources has grown exponentially. Massive Open Online Courses (MOOCs), video tutorials, research papers, and technical blogs are accessible to anyone with an internet connection. However, this wealth of information has introduced a new educational challenge: cognitive overload and unstructured learning paths. E-learning platforms typically adopt a static linear curriculum that all students must follow, regardless of their prior knowledge, learning speed, or specific objectives. A beginner is forced through redundant content, while an advanced user is unable to skip foundational topics.

The AI Powered Personalised Learning Path Generator (PathAI) is designed to mitigate these inefficiencies by offering a dynamic, personalized learning framework. By collecting baseline academic information, testing the user via a short adaptive assessment, and utilizing large language models (LLMs) hosted on Groq Cloud, PathAI constructs a customized curriculum. It integrates real-world media references (YouTube video tutorials) and supports user-uploaded textbooks to create a cohesive study roadmap, fostering structured self-education and improving retention through automated checkpoint evaluations.

1.2 Project Category

This project falls under the category of Web-Based Application and Artificial Intelligence Integration. It combines modern full-stack web engineering principles (FastAPI RESTful web service and React.js frontend library) with cloud-hosted Generative AI engines (Groq API using the Llama 3.3 70B Versatile model) and a cloud relational database management system (Supabase PostgreSQL) to automate curriculum generation and user assessment.

1.3 Objectives

The primary objectives of the PathAI platform are:

1. To generate custom, personalized learning paths based on onboarding profiles and goal criteria.

2. To evaluate the student's baseline knowledge using an initial assessment quiz to identify the correct starting level.
3. To structure learning paths into modular chapters with progressive difficulty, specific descriptions, and curated references.
4. To integrate the YouTube Data API v3 to fetch and embed high-quality video tutorials corresponding to each topic dynamically.
5. To implement Supabase Storage, allowing users to upload and link custom PDF textbooks or course notes alongside their roadmap.
6. To reinforce student learning by generating checkpoint quizzes at the end of each module using Generative AI.
7. To provide an interactive, animated progress-tracking dashboard with modern, responsive styling including dark mode support.

1.4 Scope of the Problem

Self-directed education is critical for modern professional development, especially in fast-evolving fields like Computer Science and Engineering. However, learners face significant challenges: identifying reliable resources, establishing a logical learning order, and validating whether they have actually mastered a topic. The scope of this project is to build a web platform that automates these tasks. It addresses academic and career milestones (Job Placement, Skill Building, Exam Preparation, and Competitive Coding) for college students, helping them navigate complex technical subjects by providing a single unified dashboard with curated text and video resources.

1.5 Identification of Need

Modern students learn differently; some prefer visual content (videos), others prefer detailed reading materials (PDFs), and almost all need continuous feedback to understand their weaknesses. Traditional e-learning platforms are closed ecosystems that do not allow students to bring external textbooks or combine multiple media platforms (such as YouTube and PDFs) into a single cohesive timeline. There is therefore a strong need for an open, AI-driven learning generator that bridges the gap between textbook materials, YouTube tutorials, and online evaluation quizzes — mapping them onto a unified, progress-tracking dashboard.

1.6 Existing System

The existing e-learning ecosystem consists of traditional MOOC platforms (Coursera, edX, Udemy), video-sharing platforms (YouTube), and physical university courses. In these systems, curriculum design is static. A professor or course creator designs a syllabus, and all registered students must follow the identical sequence of lectures, quizzes, and projects. While platforms like YouTube are free and comprehensive, they lack any form of testing, structured progress tracking, or personalization based on user competency level. Students must search for tutorials manually, leading to distraction and fragmented knowledge acquisition.

1.7 Limitations of the Existing System

The key limitations of existing platforms include:

- **Static Learning Path:** No customization for student speed, background, or goals. Every user follows the same curriculum regardless of prior knowledge.
- **Lack of Baseline Assessment:** No diagnostic tests are conducted to place students at the correct module. Learners either repeat known content or skip foundational concepts.
- **Fragmented Content:** Students must navigate multiple websites to find relevant videos and textbooks, leading to disorganized study sessions.
- **Passive Learning:** The absence of interactive self-assessments means students cannot verify whether they have understood a topic before proceeding.
- **Disorganized Progress Tracking:** No centralized mechanism exists to monitor completion status across different resource types and platforms.

1.8 Proposed System

The proposed system, PathAI, resolves these limitations by implementing a smart, dynamic curriculum compiler. When a student registers, they complete an onboarding form detailing their study area, baseline experience, and primary goal. The system immediately generates a 10-question multiple-choice initial assessment to evaluate actual knowledge. The score is processed by the FastAPI backend service, which computes the skill level and triggers a prompt to the Groq API. Using the Llama 3.3 70B model, Groq returns a complete, structured JSON learning path containing 4-6

modules and 15-25 topics, calibrated to the student's competency level (Beginner, Intermediate, or Advanced).

The generated path is saved to the Supabase PostgreSQL database and rendered on an interactive React dashboard. For each topic, the backend fetches a relevant YouTube tutorial via the YouTube Data API v3. The student can mark topics complete, upload and view reference PDFs, and take a 5-question checkpoint quiz at the end of each module before proceeding. This creates a closed-loop, adaptive, and outcome-driven educational system.

1.9 Unique Features of the System

- **LLM-Powered Personalization:** Curriculums are generated dynamically in real-time, tailored to the exact onboarding goal, study field, and assessed skill level.
- **Diagnostic Assessment Placement:** Automated, AI-generated MCQ quiz places the student into Beginner, Intermediate, or Advanced tracks objectively based on quiz performance.
- **Integrated YouTube Media Search:** Real-time YouTube Data API v3 integration fetches the single most relevant tutorial video per topic automatically.
- **Supabase Storage Integration:** Users can upload and reference custom textbook PDFs directly within their active learning roadmap sidebar.
- **Module Checkpoint Gatekeeping:** A 5-question AI-generated review quiz validates module mastery before the next module is unlocked.
- **Animated SVG Roadmap Dashboard:** An interactive, animated SVG timeline visualizes path progress with glassmorphic styling and real-time completion tracking.

Chapter 2: Requirement Analysis & SRS

2.1 Feasibility Study

A feasibility study was conducted to assess the viability of developing the PathAI platform. The project was evaluated across three core feasibility dimensions: Technical, Economic, and Operational.

2.1.1 Technical Feasibility

PathAI is technically feasible. The core libraries and frameworks — FastAPI, React.js, and the Supabase Client SDK — are mature, well-documented, and widely adopted in production environments. The Groq API provides high-speed inference of the Llama 3.3 70B model, returning structured JSON responses within seconds, which is crucial for a smooth user experience. Integration of YouTube search and Supabase PDF storage is handled via standard REST APIs, requiring no complex custom protocols. Development can be carried out on standard consumer hardware, and deployment can be accomplished using scalable free-tier cloud platforms.

2.1.2 Economic Feasibility

The operational costs of PathAI are extremely low. Supabase offers a generous free tier for PostgreSQL hosting and file storage. Groq Cloud provides highly cost-effective API access with a free daily quota sufficient for university-scale usage. The YouTube Data API v3 provides 10,000 free quota units per day, with caching in the database preventing redundant calls. All development tools — VS Code, Git, Python, and Node.js — are open-source and free of charge. The project is economically viable as it requires no expensive server hardware or paid software licenses.

2.1.3 Operational Feasibility

Operational feasibility is high because the platform is designed around a user-centric, intuitive interface. The onboarding process is straightforward and requires no prior technical knowledge from the student. The dashboard provides clear, real-time progress visualization. The system can be accessed from any modern web browser on desktop or mobile devices, ensuring high adoption rates among students.

2.2 Software Requirement Specification (SRS)

2.2.1 Data Requirement

The database must persist user onboarding data, initial quiz results, generated learning paths, roadmap topic details, checkpoint quiz results, and references to uploaded PDFs. The system uses Supabase PostgreSQL, enforcing referential integrity via foreign key constraints and cascading deletes when a user profile is removed. Row Level Security (RLS) policies ensure that users can only access their own data. The full schema is defined in `database/schema.sql`.

2.2.2 Functional Requirements

PathAI must support the following functional requirements:

- **User Authentication:** Students can sign up and log in securely via Supabase Auth, which provides JWT-based session management.
- **Onboarding Form:** The system must collect the student's field of study, primary goal, and self-assessed experience level before proceeding to assessment.
- **Diagnostic Quiz Engine:** Generates exactly 10 multiple-choice questions matching the study field and skill level, and evaluates the student's placement tier.
- **Roadmap Generation:** Requests and parses a structured JSON curriculum from the Groq API based on the combined onboarding profile and quiz score.
- **Resource Integration:** Automatically searches and attaches a relevant YouTube Data API result to each roadmap topic upon path generation.
- **Document Upload:** Allows the user to upload course-related PDF files to a Supabase Storage bucket and lists them in a side panel on the dashboard.
- **Progress Tracker:** Allows the user to mark individual topics as complete, updating real-time completion statistics and streak counters.
- **Checkpoint Evaluations:** Triggers a 5-question AI-generated MCQ quiz at the end of each module to assess mastery before unlocking the next module.

2.2.3 Performance Requirements

- Page load times must be under 2 seconds for previously cached learning paths.
- The full learning path generation cycle (FastAPI to Groq API to Supabase write) must complete within 8 seconds under normal network conditions.

- Each YouTube Data API search request must complete in under 1 second, with results cached in the `path_topics` table to prevent duplicate API calls.
- The system must support concurrent API requests from a minimum of 50 simultaneous active learners.

2.2.4 Dependability & Maintainability Requirements

- **Availability:** The platform must maintain 99% uptime through cloud hosting on Supabase and Vercel.
- **Exception Handling:** All backend service failures must be caught and return standard HTTP error codes (400, 404, 500) with descriptive messages.
- **Modular Code:** Code is organized into isolated FastAPI routers (`onboarding`, `quiz`, `learning_path`, `youtube`, `pdfs`) to simplify debugging and future extension.

2.2.5 Security Requirements

- **Supabase Row Level Security (RLS)** policies must strictly prevent any user from reading or modifying another user's onboarding data, learning paths, or quiz scores.
- **Supabase Storage Policies** must enforce that users can only upload files to their own personal directory within the 'user-pdfs' storage bucket.
- **JWT Token Verification:** All protected backend endpoints must validate Supabase-issued JWT access tokens before processing any request or returning user data.

2.2.6 Look and Feel Requirements

- The UI must be modern and visually polished, using glassmorphic card backgrounds, dark/light theme switching, and smooth framer-motion animations.
- The design must be fully responsive, supporting seamless use across desktop computers, tablets, and mobile smartphones.

2.3 Validation

Data validation is enforced at both the frontend and backend layers. On the onboarding page, form validation prevents empty field submissions and restricts invalid inputs. The quiz engine verifies that users have submitted an answer for every question before scoring. The FastAPI backend validates all incoming request payloads using Pydantic schemas, returning 422 Unprocessable Entity responses for malformed inputs. Supabase database check constraints enforce that skill level values are strictly limited to 'beginner', 'intermediate', or 'advanced'.

2.4 Expected Hurdles

8. JSON Response Consistency: LLMs can sometimes append markdown code fences (e.g. ````json...````) or conversational text around their structured output. To handle this, the backend implements regular expression stripping and a three-attempt retry loop in `groq_service.py` to ensure stable JSON parsing.
9. Groq API Rate Limits: Under high concurrent user traffic, Groq API endpoints may return HTTP 429 Rate Limit errors. The system mitigates this by caching generated learning paths in the database, preventing redundant generation calls on page refresh.
10. YouTube API Quota Management: The free YouTube Data API v3 tier provides 10,000 quota units per day. Caching video metadata directly in the `path_topics.youtube_data` column prevents duplicate API calls for topics that have already been fetched.

2.5 SDLC Model

The project was developed using the Agile Software Development Life Cycle (SDLC) model. Development was divided into five 2-week sprints, each focusing on building and testing an incremental set of components: Sprint 1 covered database schema creation and authentication setup; Sprint 2 covered FastAPI integration with the Groq API; Sprint 3 covered the React onboarding flow and quiz dashboard; Sprint 4 covered the interactive roadmap visualizer and YouTube API integration; Sprint 5 covered checkpoint quizzes, PDF uploads, and system optimization. This iterative approach allowed bugs to be identified early, user experience to be refined, and stable performance to be ensured throughout development.

Chapter 3: System Design

3.1 Design Approach

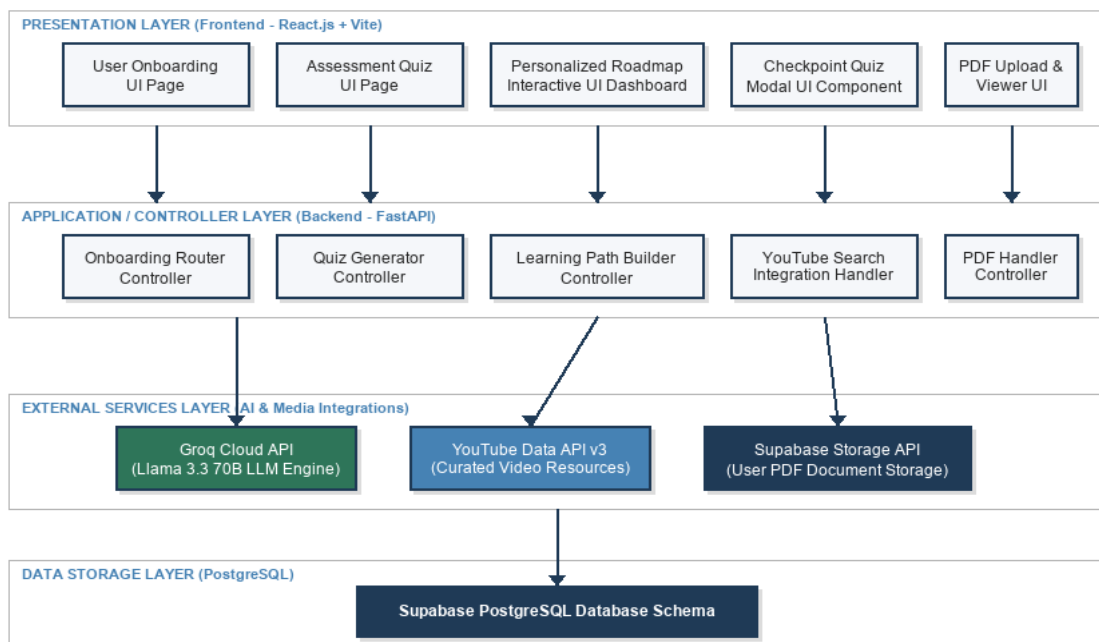
PathAI adopts an Object-Oriented Design (OOD) approach throughout both its backend and frontend layers. The data layer is structured around well-defined relational tables that correspond directly to application models. On the backend, FastAPI controllers and route modules encapsulate logic for specific domain areas including onboarding, quiz generation, learning path management, YouTube search, and PDF handling. On the frontend, React.js uses a component-based model where state is managed locally or globally via Context APIs (AuthContext, ThemeContext), promoting reusability, separation of concerns, and clean lifecycle management.

3.2 System Architecture

The architecture of PathAI follows a client-server, three-tier web layout. The Presentation Layer (React.js + Vite) runs in the user's browser, communicating with the Application Layer (FastAPI backend) via secure HTTPS REST calls. The backend orchestrates all business logic, interfaces with the Supabase PostgreSQL database, and routes external requests to the Groq AI and YouTube Data API services.

Figure 3.1: Multi-layered System Architecture Diagram of the PathAI Platform

PathAI System Architecture



3.3 Structured Analysis & UML Design

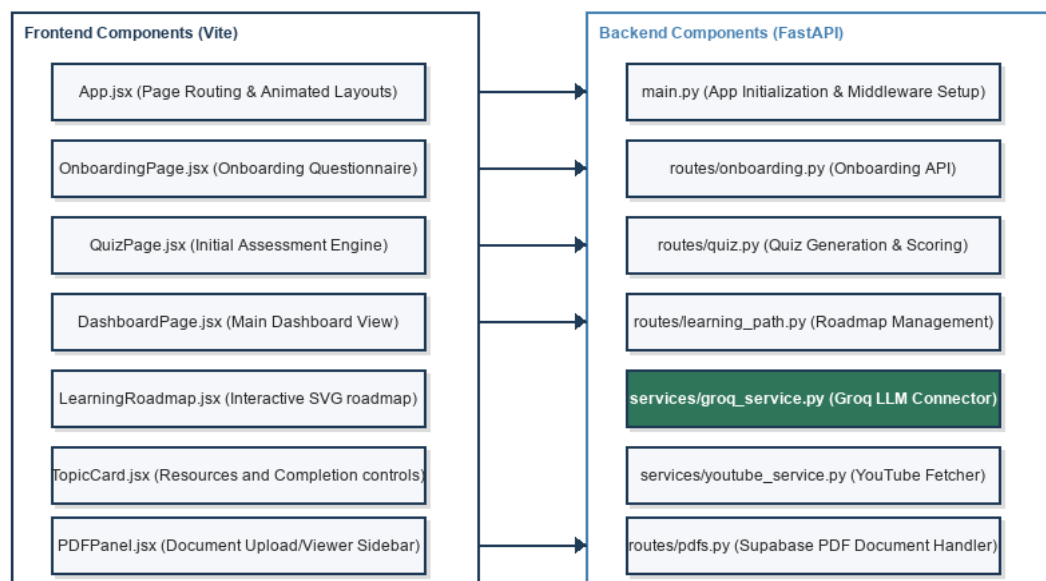
To model the structure, behavior, and physical deployment of PathAI, a complete set of Unified Modeling Language (UML) diagrams was designed. Each diagram captures a different perspective of the system.

3.3.1 Component Diagram

The Component Diagram illustrates the modular organization of both the React.js frontend modules and the FastAPI backend packages, detailing how they communicate via REST API endpoints. Frontend modules include OnboardingPage.jsx, QuizPage.jsx, DashboardPage.jsx, LearningRoadmap.jsx, TopicCard.jsx, and PDFPanel.jsx. Backend packages include routes for onboarding, quiz generation, learning path management, YouTube search, and PDF handling, all served through app/main.py.

Figure 3.2: Component Diagram of PathAI Platform

Component Diagram of PathAI Platform

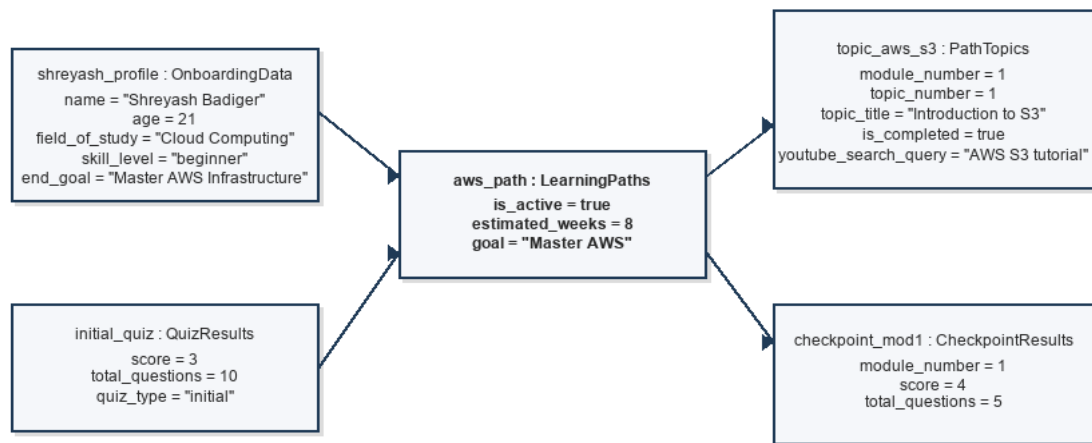


3.3.2 Object Diagram

The Object Diagram shows the actual instance variables and their runtime relationships using an example path generation transaction for a student named Shreyash. It demonstrates how an OnboardingData object, QuizResults object, LearningPaths record, and associated PathTopics instances are created and linked during a single session.

Figure 3.3: Object Diagram Demonstrating Runtime Instance Values

Object Diagram: Example Runtime Instances of Models

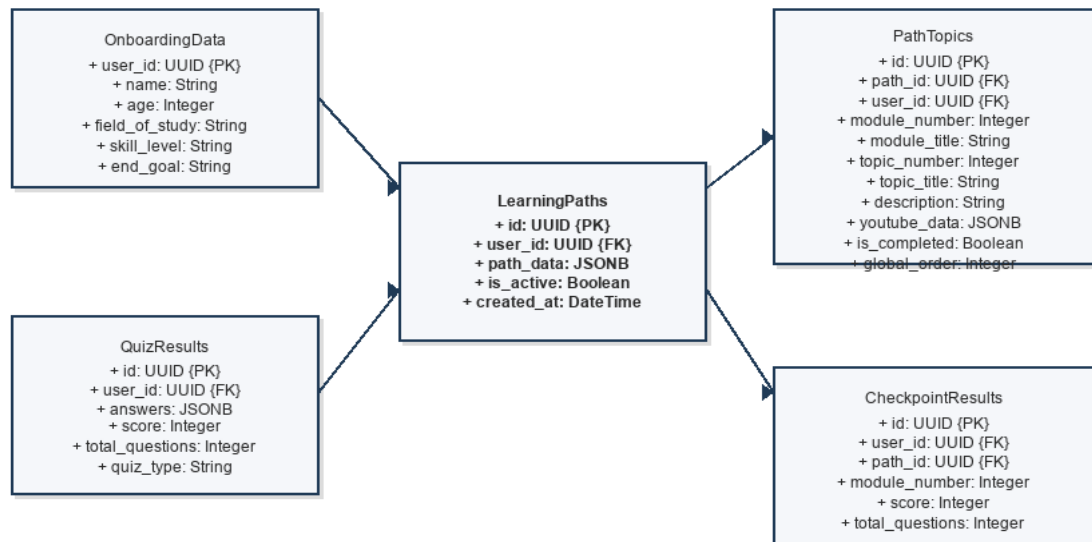


3.3.3 Class Diagram

The Class Diagram defines all database tables in the Supabase PostgreSQL instance and illustrates their primary key (PK) and foreign key (FK) associations. The five core tables — `onboarding_data`, `learning_paths`, `path_topics`, `quiz_results`, and `checkpoint_results` — are shown with their field types and referential relationships.

Figure 3.4: Class Diagram Modeling Database Models and Referential Relationships

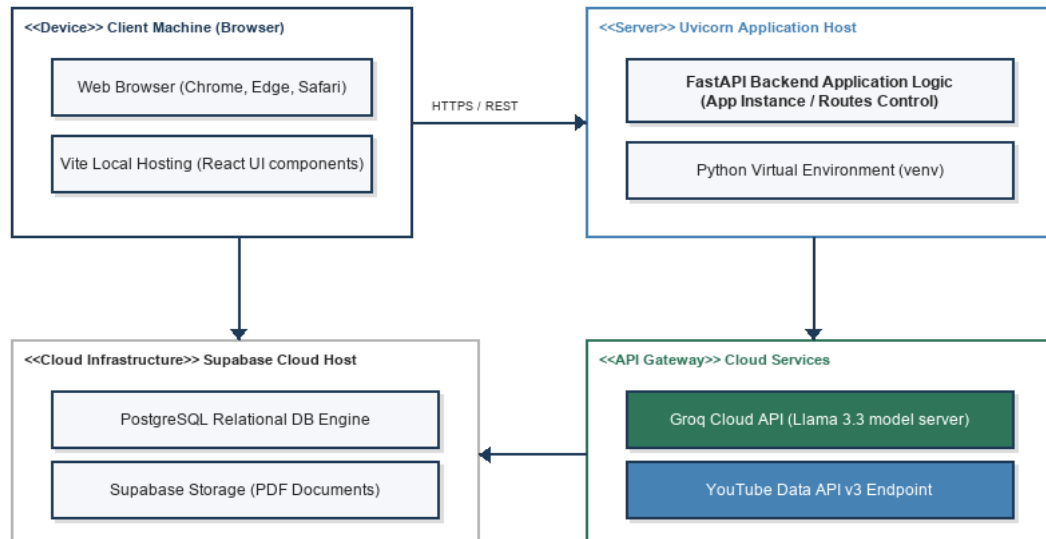
Class Diagram: Data Models and Supabase Relationships



3.3.4 Deployment Diagram

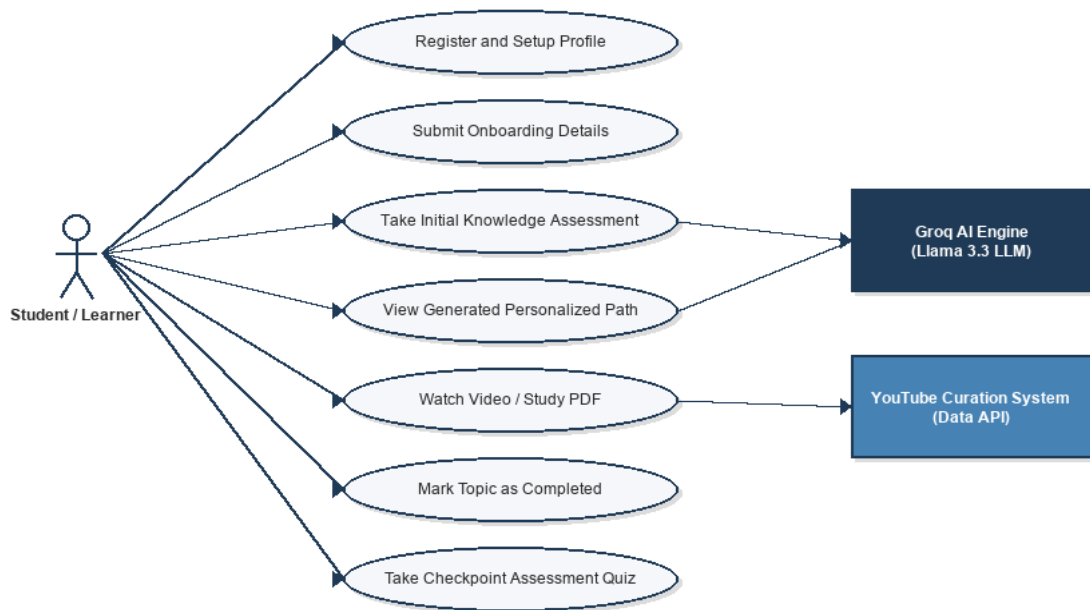
The Deployment Diagram describes the physical nodes, hosting platforms, and communication protocols that comprise the running web application. The Client Machine node runs the React.js Vite application in a web browser. The Server node (Uvicorn ASGI host) runs the FastAPI Python application. The Supabase Cloud Host node provides the PostgreSQL relational database engine and the Storage bucket for user PDF documents. External API services include the Groq Cloud API and the YouTube Data API v3 endpoint.

Figure 3.5: Deployment Diagram Showing Hardware Nodes and Network Interfaces

Deployment Diagram of the PathAI Platform**3.3.5 Use Case Diagram**

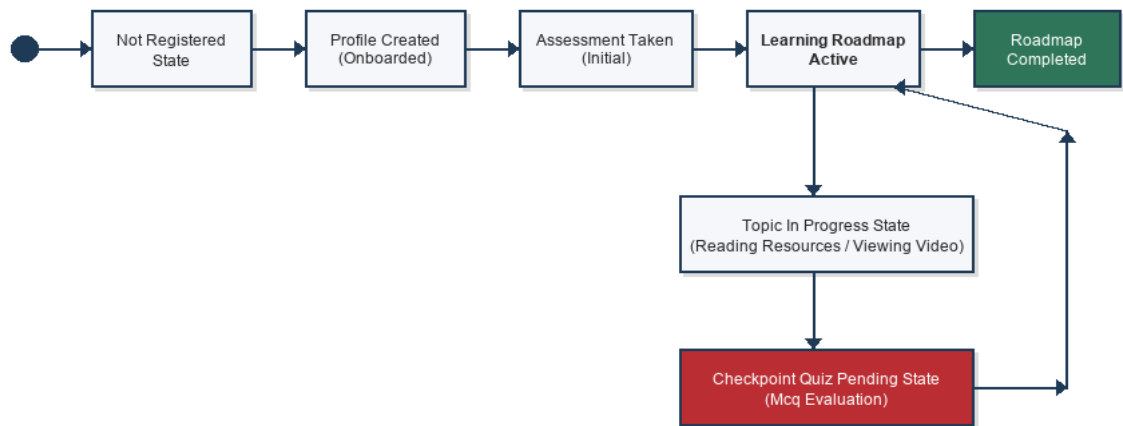
The Use Case Diagram defines the interactions of the Student/Learner role with the platform's core use cases and external backend services. The seven primary use cases are: Register and Setup Profile; Submit Onboarding Details; Take Initial Knowledge Assessment; View Generated Personalized Path; Watch Video or Study PDF; Mark Topic as Completed; and Take Checkpoint Assessment Quiz. The Groq AI Engine and YouTube Curation System appear as external actors that support the learning path generation and video resource use cases respectively.

Figure 3.6: Use Case Diagram Modeling Student Learner Interactions

Use Case Diagram for Student Learner Role**3.3.6 State Diagram**

The State Transition Diagram illustrates the lifecycle phases of a student's learning path from registration through path completion. The states are: Not Registered; Profile Created (Onboarded); Assessment Taken (Initial Quiz); Learning Roadmap Active; Topic In Progress (Reading Resources / Viewing Video); Checkpoint Quiz Pending (MCQ Evaluation); and Roadmap Completed. Transitions between states are triggered by user actions such as completing the onboarding form, submitting quiz answers, and marking topics as complete.

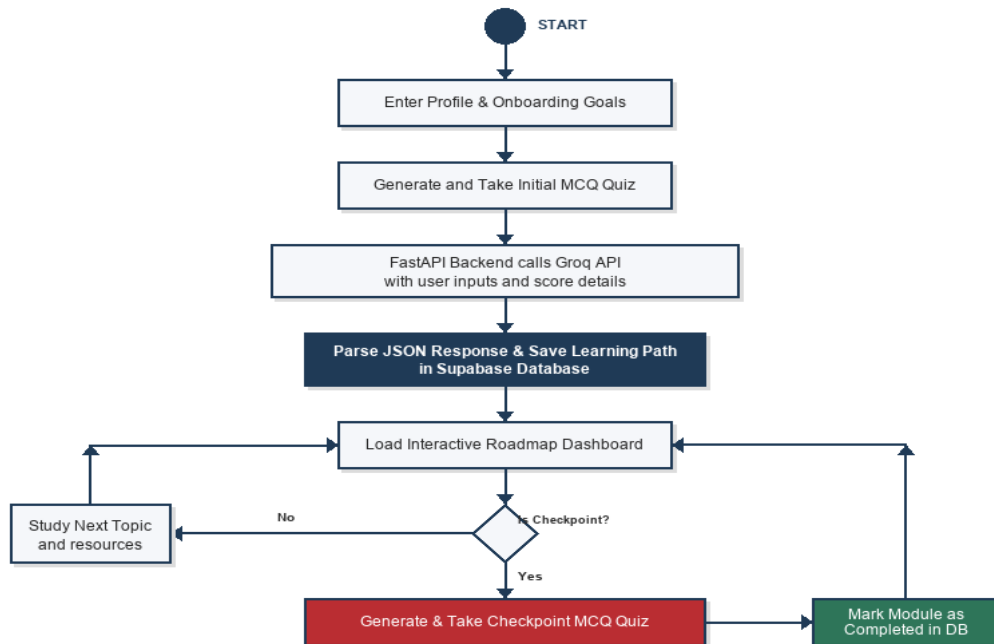
Figure 3.7: State Transition Diagram Showing Path Lifecycle Phases

State Transition Diagram: Onboarding and Path Completion**3.3.7 Activity Diagram**

The Activity Diagram models the complete step-by-step logic of the PathAI system from the moment the student opens the application to final roadmap completion. The flow begins with profile and goal entry, proceeds through AI-generated quiz assessment, triggers the FastAPI-to-Groq path generation cycle, and branches at each checkpoint decision point to either continue to the next topic or generate a checkpoint quiz for module evaluation.

Figure 3.8: Activity Diagram Showing Sequential System Workflows

Activity Diagram: Learning Path Generation & Tracking Workflow

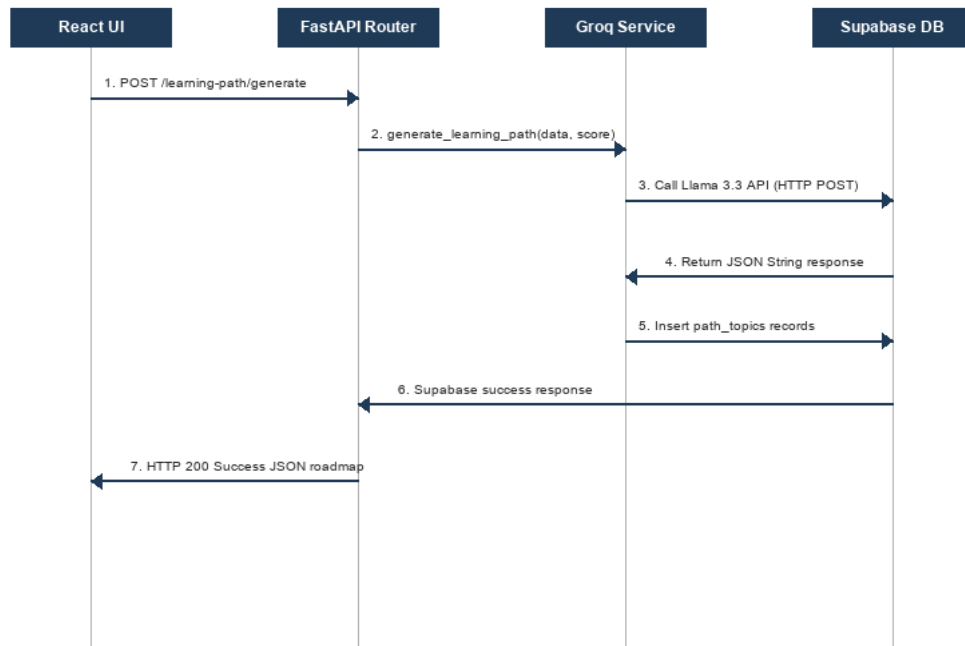


3.3.8 Sequence Diagram

The Sequence Diagram captures the sequential message transactions between the React UI, FastAPI Router, Groq Service, and Supabase Database during the learning path generation operation. The seven-step interaction is: (1) React UI sends POST /learning-path/generate; (2) FastAPI router invokes generate_learning_path(); (3) Groq Service calls the Llama 3.3 API; (4) Groq returns a structured JSON string response; (5) FastAPI inserts path_topics records into Supabase; (6) Supabase returns a success response; (7) FastAPI returns HTTP 200 with the full path_data JSON to the React UI.

Figure 3.9: Sequence Diagram Demonstrating Real-Time Message Flows

Sequence Diagram: Learning Path Generation Transaction Sequence

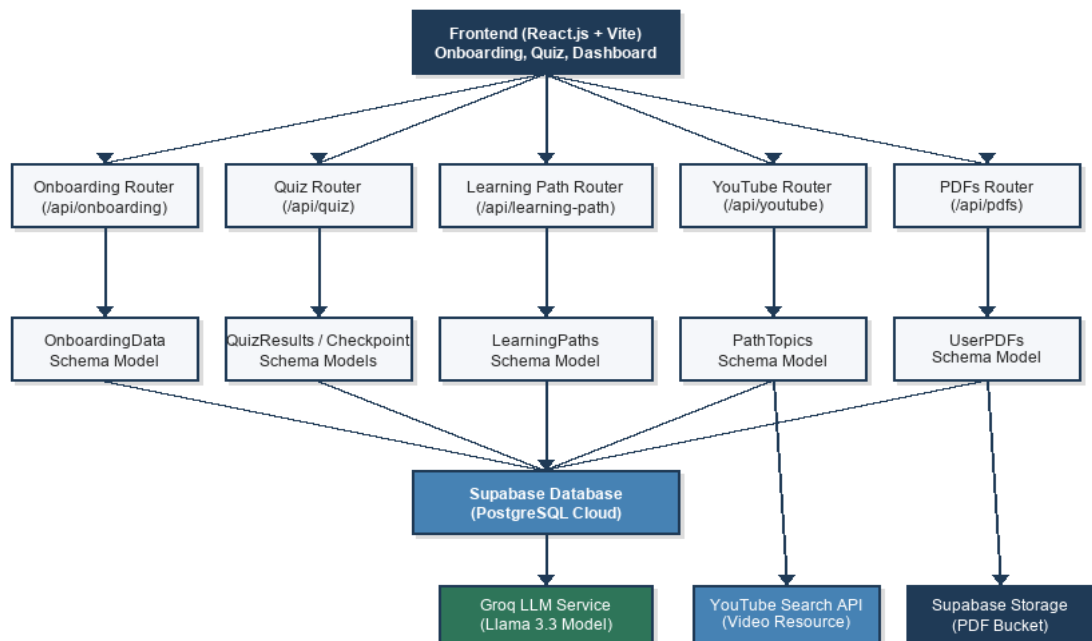


3.4 Interface Relationship & Dependencies

The system enforces clean, unidirectional dependencies throughout. The React UI depends strictly on FastAPI controllers via HTTP REST calls. FastAPI controllers depend on Supabase client models for database operations and on the Groq Service for AI calls. The Groq Service and YouTube Service are stateless, external dependency leaves of the architecture. API calls are structured to avoid any circular dependencies. This design ensures that each layer can be independently tested, replaced, or scaled.

Figure 3.10: Interface Relationship and Module Dependencies Chart

Interface Relationship and Dependencies Between Modules



3.5 Database Design

The relational database schema is structured into normalized tables in Supabase PostgreSQL. Row Level Security (RLS) policies are applied to all tables to enforce per-user data isolation. The following data dictionaries describe each table's fields, types, and constraints.

Table 3.1: onboarding_data — Data Dictionary

Field	Type	Null	Key	Default	Description
id	uuid	NO	PRI	uuid_generate_v4()	Primary Key identifying the onboarding record.
user_id	uuid	NO	UNI	—	FK → auth.users(id). One record per user.
name	text	NO	—	—	Full name of the student.
age	integer	YES	—	—	Age of the student (optional).

Field	Type	Null	Key	Default	Description
field_of_study	text	NO	—	—	Field of study (e.g. Cloud Computing).
skill_level	text	NO	—	—	beginner intermediate advanced.
end_goal	text	NO	—	—	Primary career or learning objective.
created_at	timestampz	YES	—	now()	Record creation timestamp.

Table 3.2: learning_paths — Data Dictionary

Field	Type	Null	Key	Default	Description
id	uuid	NO	PRI	uuid_generate_v4()	Primary Key identifying the learning path.
user_id	uuid	NO	MUL	—	FK → auth.users(id). Multiple paths per user.
path_data	jsonb	NO	—	—	Full JSON curriculum (modules and topics).
is_active	boolean	YES	—	true	Flags the student's currently active roadmap.
created_at	timestampz	YES	—	now()	Path creation timestamp.

Table 3.3: path_topics — Data Dictionary

Field	Type	Null	Key	Default	Description
id	uuid	NO	PRI	uuid_generate_v4()	Primary Key for the topic record.
path_id	uuid	NO	MUL	—	FK → learning_paths(id) ON DELETE CASCADE.

Field	Type	Null	Key	Default	Description
user_id	uuid	NO	MUL	—	FK → auth.users(id) ON DELETE CASCADE.
module_number	integer	NO	—	—	Module order index (1-based).
module_title	text	NO	—	—	Title of the parent module.
topic_number	integer	NO	—	—	Topic order within its module (1-based).
topic_title	text	NO	—	—	Title of the specific learning topic.
description	text	YES	—	—	2-3 sentence topic description.
youtube_data	jsonb	YES	—	—	Cached YouTube API metadata.
is_checkpoint	boolean	YES	—	false	True for the last topic of each module.
is_completed	boolean	YES	—	false	True when the student marks topic done.
global_order	integer	NO	—	—	Sequential index across the entire roadmap.

Table 3.4: quiz_results — Data Dictionary

Field	Type	Null	Key	Default	Description
id	uuid	NO	PRI	uuid_generate_v4()	Primary Key for the quiz result.
user_id	uuid	NO	MUL	—	FK → auth.users(id).
answers	jsonb	YES	—	—	JSON: questions, options, and correct answers.
score	integer	NO	—	—	Number of correct answers.
total_questions	integer	NO	—	—	Total questions in the quiz.

Field	Type	Null	Key	Default	Description
quiz_type	text	NO	—	'initial'	'initial' or 'checkpoint'.
created_at	timestampz	YES	—	now()	Quiz submission timestamp.

Table 3.5: checkpoint_results — Data Dictionary

Field	Type	Null	Key	Default	Description
id	uuid	NO	PRI	uuid_generate_v4()	Primary Key.
user_id	uuid	NO	MUL	—	FK → auth.users(id).
path_id	uuid	NO	MUL	—	FK → learning_paths(id).
module_number	integer	NO	—	—	Module that was evaluated.
module_title	text	YES	—	—	Title of the evaluated module.
score	integer	NO	—	—	Number of correct answers.
total_questions	integer	NO	—	—	Total questions (always 5).
created_at	timestampz	YES	—	now()	Checkpoint submission timestamp.

3.6 User Interface Design

The user interface is built with React.js and Tailwind CSS, applying a consistent glassmorphic design language with smooth framer-motion animations across all pages. The color palette uses a dark background with semi-transparent frosted glass card components, accented with blue and purple highlights. Typography uses the Inter font family for clean, modern readability. Dark mode and light mode are both supported, with the preference persisted in React context.

The six primary pages and their design characteristics are:

- **Authentication Page (AuthPage.jsx):** Glassmorphic card with sign-up and log-in tabs, form validation feedback, and a smooth fade-in animation on mount.
- **Onboarding Form (OnboardingPage.jsx):** Multi-field form collecting field of study, age, skill level, and end goal. Input validation prevents empty submissions.
- **Assessment Quiz Page (QuizPage.jsx):** Renders 10 AI-generated MCQ questions with animated question transitions, a progress indicator, and a final submit flow.
- **Roadmap Dashboard (DashboardPage.jsx):** The core interface featuring an animated SVG timeline, expandable topic cards with YouTube video embedding, and a PDF sidebar panel.
- **Checkpoint Quiz Modal:** A modal overlay triggered at the completion of each module, presenting 5 AI-generated review questions before unlocking the next module.
- **PDF Upload Sidebar (PDFPanel.jsx):** Allows drag-and-drop or file-picker PDF uploads to Supabase Storage, with a listing of previously uploaded documents.

3.7 REST API Endpoints

Table 3.6 below lists all key REST API endpoints exposed by the FastAPI backend.

Table 3.6: REST API Endpoint Reference

Endpoint	Method	Parameters	Response	Description
/health	GET	None	<code>{"status": "ok"}</code>	Health check — confirms API is running.
/api/onboarding	POST	OnboardingData (JSON)	<code>{"success": true}</code>	Saves onboarding profile to DB.

Endpoint	Method	Parameters	Response	Description
/api/quiz/generate	POST	field, level, goal (JSON)	Quiz JSON Array (10 questions)	Generates initial 10-question MCQ assessment via Groq.
/api/learning-path/generate	POST	onboarding_data, quiz_score, quiz_total	{"success": true, "path_data": ...}	Triggers Groq LLM path generation and writes to DB.
/api/learning-path	GET	None (JWT auth)	{"path": ..., "topics": ...}	Fetches active path and all topic completion states.
/api/learning-path/all	GET	None (JWT auth)	{"paths": [...]}	Returns all paths for the authenticated user.
/api/learning-path/{id}/activate	PATCH	path_id (URL)	{"success": true}	Sets the specified path as the active roadmap.
/api/learning-path/topics/{id}/complete	PATCH	topic_id (URL)	{"success": true}	Marks a topic as completed with timestamp.
/api/youtube/search	POST	query (string)	YouTube Video Metadata	Queries YouTube Data API v3 and returns video metadata.
/api/pdfs/upload	POST	File (Multipart)	{"file_url": ...}	Uploads PDF to Supabase Storage and registers in DB.

Chapter 4: Implementation, Testing & Maintenance

4.1 Tools and Technologies Used

The following technologies were selected and used to build the PathAI platform:

- FastAPI (Python 3.11): A modern, high-performance ASGI web framework selected for its speed comparable to Node.js and Go, automatic OpenAPI/Swagger UI documentation generation, and type-safe request validation using Pydantic schemas.
- React.js (Vite): A component-based frontend library built with the Vite build tool for ultra-fast hot module replacement during development and optimized production bundles. Framer-motion is used for all page transition and component animations.
- Supabase (PostgreSQL): An open-source Firebase alternative providing a managed PostgreSQL database engine, a built-in Supabase Auth JWT authentication system, a SQL editor, and S3-compatible file Storage Buckets — all accessible via a unified Python and JavaScript SDK.
- Groq Cloud (Llama 3.3 70B Versatile): A high-performance LLM inference hosting service utilizing specialized Language Processing Units (LPUs), providing sub-second response times for structured JSON generation tasks including quiz creation, learning path compilation, and checkpoint quiz generation.
- YouTube Data API v3: Integrated to search for and retrieve the single most relevant tutorial video for each roadmap topic automatically, with video metadata cached in the `path_topics` table to conserve daily quota units.
- Tailwind CSS: A utility-first CSS framework used for responsive layout design, spacing, and glassmorphic component styling throughout the frontend.

4.2 Coding Standards

To ensure maintainability, readability, and ease of collaboration, strict coding standards were enforced across both repository layers. All Python backend code follows PEP 8 standards, utilizing comprehensive type hints and Pydantic model schemas for request and response parameter validation. All FastAPI route handlers are implemented as

async functions to maximize throughput. Frontend JavaScript follows ES6+ guidelines, using arrow functions, destructuring, and the React Hooks API (useState, useEffect, useContext). ESLint rules enforce consistent component structure and prevent common anti-patterns.

4.3 Testing Techniques and Test Plans

A multi-tiered testing strategy was implemented to verify system behavior across all layers. Unit Testing verified individual FastAPI route handlers and utility functions (such as the `_clean_json` parser in `groq_service.py`) in isolation. Integration Testing verified end-to-end communication between the React frontend, FastAPI REST endpoints, and Supabase database CRUD operations. User Acceptance Testing (UAT) validated all onboarding workflows, quiz grading logic, YouTube fetch integration, roadmap generation, checkpoint evaluation, and PDF upload flows against the requirements defined in the SRS.

4.4 Executable Code Listings

This section presents the actual, complete source code for the core modules of the PathAI platform, reflecting the files in the production workspace.

Listing 4.1: database/schema.sql — SQL Database Schema

```
create extension if not exists "uuid-oss";

create table if not exists onboarding_data (
  id uuid primary key default uuid_generate_v4(),
  user_id uuid references auth.users(id) on delete cascade unique,
  name text not null,
  age int,
  field_of_study text not null,
  skill_level text not null check (skill_level in
('beginner', 'intermediate', 'advanced')),
  end_goal text not null,
  created_at timestamptz default now()
);

create table if not exists learning_paths (
  id uuid primary key default uuid_generate_v4(),
  user_id uuid references auth.users(id) on delete cascade,
  path_data jsonb not null,
  is_active boolean default true,
  created_at timestamptz default now()
);

create table if not exists path_topics (
  id uuid primary key default uuid_generate_v4(),
  path_id uuid references learning_paths(id) on delete cascade,
  user_id uuid references auth.users(id) on delete cascade,
  module_number int not null, module_title text not null,
  topic_number int not null, topic_title text not null,
```

```

description text,          youtube_data jsonb,
is_checkpoint boolean default false,
is_completed boolean default false,
global_order int not null,  completed_at timestamptz
);

-- Row Level Security
alter table onboarding_data enable row level security;
alter table learning_paths  enable row level security;
alter table path_topics     enable row level security;

create policy "own_onboarding" on onboarding_data
  for all using (auth.uid() = user_id);
create policy "own_paths" on learning_paths
  for all using (auth.uid() = user_id);
create policy "own_topics" on path_topics
  for all using (auth.uid() = user_id);

```

Listing 4.2: backend/app/main.py — FastAPI Application Entry Point

```

from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from app.config import CORS_ORIGINS
from app.routes import onboarding, quiz, learning_path, youtube, pdfs

app = FastAPI(title="AI Learning Path Generator API", version="1.0.0")

app.add_middleware(CORSMiddleware, allow_origins=CORS_ORIGINS,
                  allow_credentials=True, allow_methods=["*"], allow_headers=["*"])

app.include_router(onboarding.router, prefix="/api")
app.include_router(quiz.router, prefix="/api")
app.include_router(learning_path.router, prefix="/api")
app.include_router(youtube.router, prefix="/api")
app.include_router(pdfs.router, prefix="/api")

@app.get("/health")
def health_check():
    return {"status": "ok", "service": "AI Learning Path Generator"}

```

Listing 4.3: backend/app/services/groq_service.py — Groq LLM Integration

```

import httpx, json, re
from app.config import GROQ_API_KEY

GROQ_API_URL = "https://api.groq.com/openai/v1/chat/completions"
MODEL        = "llama-3.3-70b-versatile"

def _clean_json(content: str) -> str:
    content = content.strip()
    content = re.sub(r'^```\json\s*', '', content)
    content = re.sub(r'^```\s*', '', content)
    content = re.sub(r'\s*```$', '', content)
    return content.strip()

async def _call_groq(system_prompt, user_prompt, max_tokens=4096):
    async with httpx.AsyncClient(timeout=90.0) as client:
        resp = await client.post(GROQ_API_URL,
                                headers={"Authorization": f"Bearer {GROQ_API_KEY}"},

```

```

        json={"model": MODEL, "temperature": 0.3,
              "max_tokens": max_tokens,
              "messages": [{"role": "system", "content":
system_prompt},
                          {"role": "user", "content":
user_prompt}]})
        resp.raise_for_status()
        return resp.json()["choices"][0]["message"]["content"]

async def generate_learning_path(onboarding_data, quiz_score,
quiz_total):
    score_pct = round((quiz_score / quiz_total) * 100) if quiz_total else
0
    level = "beginner" if score_pct < 40 else
            "intermediate" if score_pct < 70 else "advanced"
    user_prompt = (
        f"Field: {onboarding_data['field_of_study']}\n"
        f"Assessed Level ({quiz_score}/{quiz_total} = {score_pct}%):
{level}\n"
        f"Goal: {onboarding_data['end_goal']}\n"
        f"Generate 4-6 modules, 3-5 topics each. Set checkpoint=true on"
        f" the last topic of every module."
    )
    for _ in range(3):
        try:
            return json.loads(_clean_json(
                await _call_groq(LEARNING_PATH_SYSTEM_PROMPT,
user_prompt, 8192)))
        except json.JSONDecodeError:
            pass
    raise Exception("Failed to parse learning path JSON after 3
attempts")

```

Listing 4.4: frontend/src/App.jsx — React Page Routing

```

import { BrowserRouter, Routes, Route, Navigate, useLocation } from
'react-router-dom'
import { AnimatePresence, motion } from 'framer-motion'
import { AuthProvider, useAuth } from './contexts/AuthContext'
import { ThemeProvider } from './contexts/ThemeContext'
import AuthPage from './pages/AuthPage'
import OnboardingPage from './pages/OnboardingPage'
import QuizPage from './pages/QuizPage'
import DashboardPage from './pages/DashboardPage'

function ProtectedRoute({ children }) {
    const { user, loading } = useAuth()
    if (loading) return <LoadingSpinner fullscreen />
    return user ? children : <Navigate to="/auth" replace />
}

function Page({ children }) {
    return (<motion.div
        initial={{ opacity: 0, y: 12, filter: 'blur(4px)' }}
        animate={{ opacity: 1, y: 0, filter: 'blur(0px)' }}
        exit={{ opacity: 0, y: -12, filter: 'blur(4px)' }}
        transition={{ duration: 0.32, ease: [0.4, 0, 0.2, 1] }}
    >{children}</motion.div>)
}

export default function App() {
    return (<ThemeProvider><AuthProvider><BrowserRouter>
        <AnimatedRoutes />
    </BrowserRouter></AuthProvider></ThemeProvider>)
}

```

Chapter 5: Results and Discussions

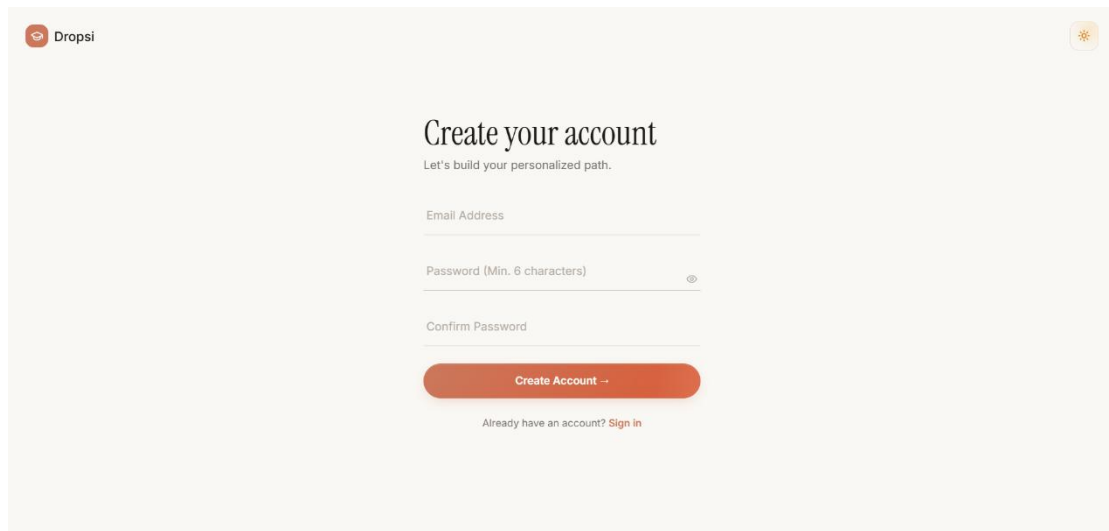
5.1 User Interface Representation

This section describes the layout, functionality, and styling of the primary user interfaces comprising the PathAI platform. All interfaces are built with a glassmorphic aesthetic using Tailwind CSS utility classes and animated using the framer-motion library for smooth, professional page transitions and component interactions.

11. Authentication Page (AuthPage.jsx): Provides secure Sign Up and Login controls inside a centered glassmorphic card. The page displays real-time form validation error messages and credential status feedback. A smooth fade-in animation is applied on mount.
12. Onboarding Form (OnboardingPage.jsx): Collects the student's field of study, age, self-assessed experience level, and primary learning goal. Input validation prevents empty form submissions and guides the user clearly through the required fields.
13. Assessment Quiz Page (QuizPage.jsx): Renders the 10 AI-generated diagnostic MCQ questions with animated question transitions. Users select an option and can navigate between questions. On final submission, a loading state indicates scoring is in progress.
14. Roadmap Dashboard (DashboardPage.jsx): The core interface of the application, displaying an animated SVG timeline representing all roadmap modules. Individual topic cards are clickable and expand to reveal the topic description, the embedded YouTube tutorial video player, and any uploaded PDF references.
15. Checkpoint Quiz Modal: A full-screen modal overlay triggered automatically when the last topic of a module is completed. It presents 5 AI-generated MCQ questions testing retention of the completed module's concepts. The next module is locked until the checkpoint is passed.
16. PDF Resource Sidebar (PDFPanel.jsx): A slide-in sidebar panel allowing students to upload reference PDF textbooks via drag-and-drop or file picker. Uploaded files are stored in the Supabase Storage 'user-pdfs' bucket and listed with a direct public URL for in-browser viewing.

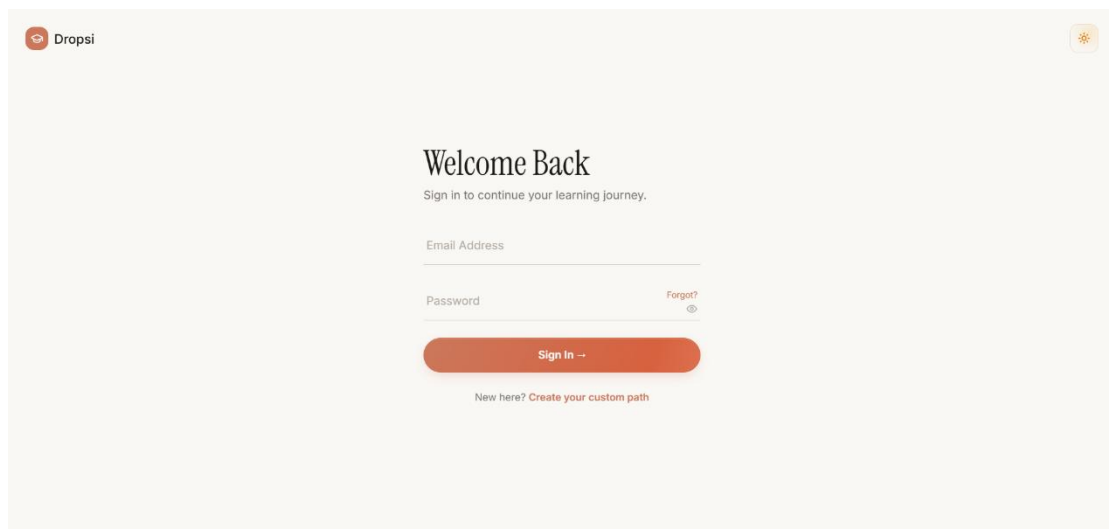
5.2 System Screenshots

Figures 5.1 through 5.7 present annotated screenshots of the PathAI platform captured from the running application at <http://localhost:5173>.



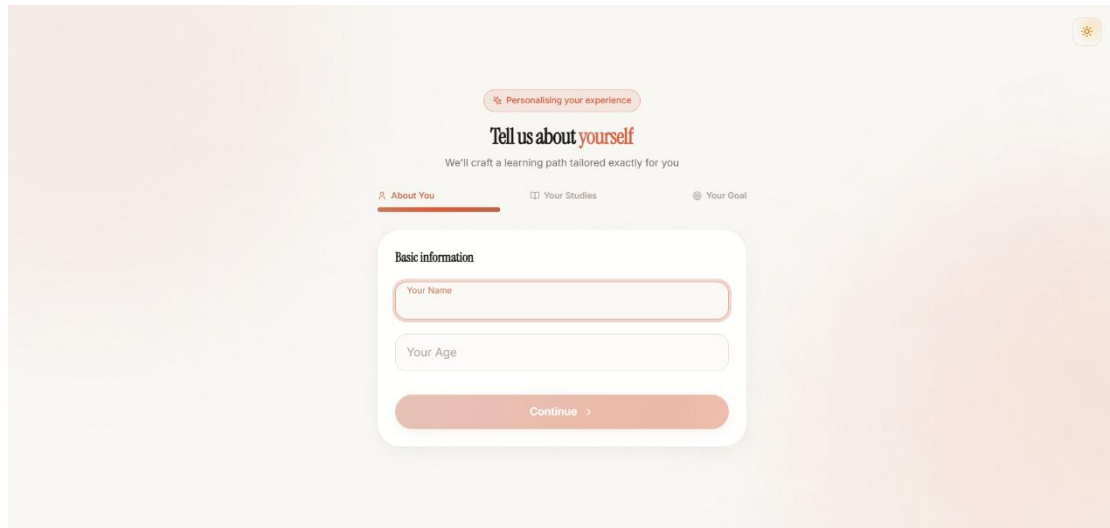
The screenshot shows a web interface for creating an account. At the top left is the 'Dropsi' logo, and at the top right is a settings icon. The main heading is 'Create your account' with the subtext 'Let's build your personalized path.' Below this are three input fields: 'Email Address', 'Password (Min. 6 characters)' (with an eye icon for toggling visibility), and 'Confirm Password'. A large orange button labeled 'Create Account --' is centered below the fields. At the bottom, a link reads 'Already have an account? Sign in'.

Figure 5.1:1 Authentication Page — Sign Up Interface



The screenshot shows a web interface for logging in. At the top left is the 'Dropsi' logo, and at the top right is a settings icon. The main heading is 'Welcome Back' with the subtext 'Sign in to continue your learning journey.' Below this are two input fields: 'Email Address' and 'Password' (with a 'Forgot?' link and an eye icon for toggling visibility). A large orange button labeled 'Sign In --' is centered below the fields. At the bottom, a link reads 'New here? Create your custom path'.

Figure 5.1:2 Authentication Page — Login Interface



Personalising your experience

Tell us about yourself

We'll craft a learning path tailored exactly for you

[About You](#)
[Your Studies](#)
[Your Goal](#)

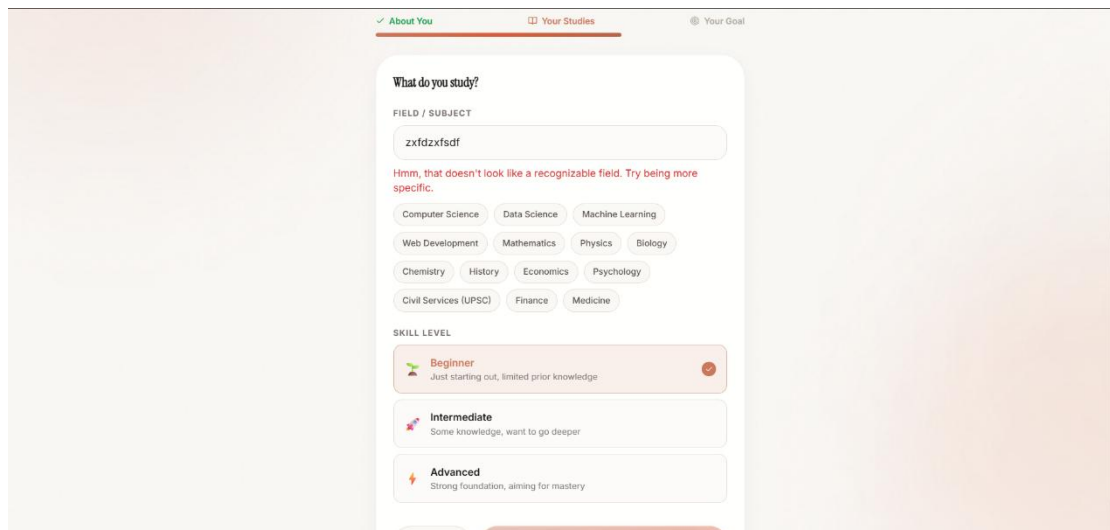
Basic information

Your Name

Your Age

[Continue >](#)

Figure 5.2: Onboarding Form — Student Profile Collection



[About You](#)
[Your Studies](#)
[Your Goal](#)

What do you study?

FIELD / SUBJECT

Hmm, that doesn't look like a recognizable field. Try being more specific.

[Computer Science](#)
[Data Science](#)
[Machine Learning](#)
[Web Development](#)
[Mathematics](#)
[Physics](#)
[Biology](#)
[Chemistry](#)
[History](#)
[Economics](#)
[Psychology](#)
[Civil Services \(UPSC\)](#)
[Finance](#)
[Medicine](#)

SKILL LEVEL

[Beginner](#) ☒ Just starting out, limited prior knowledge

[Intermediate](#) ☐ Some knowledge, want to go deeper

[Advanced](#) ☐ Strong foundation, aiming for mastery

Figure 5.2: Onboarding Form — Student Profile Collection

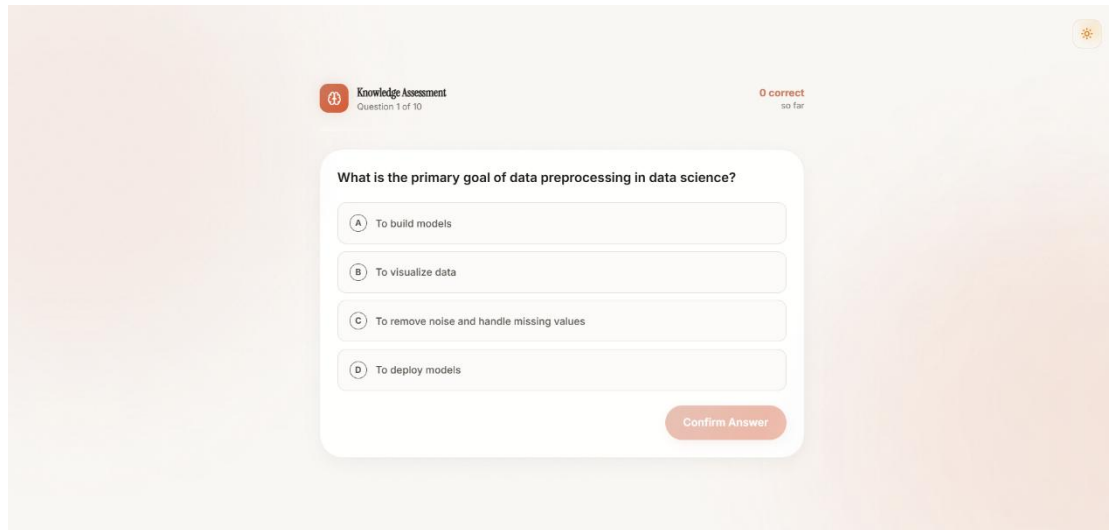


Figure 5.3: Initial Assessment Quiz — 10-Question Diagnostic MCQ

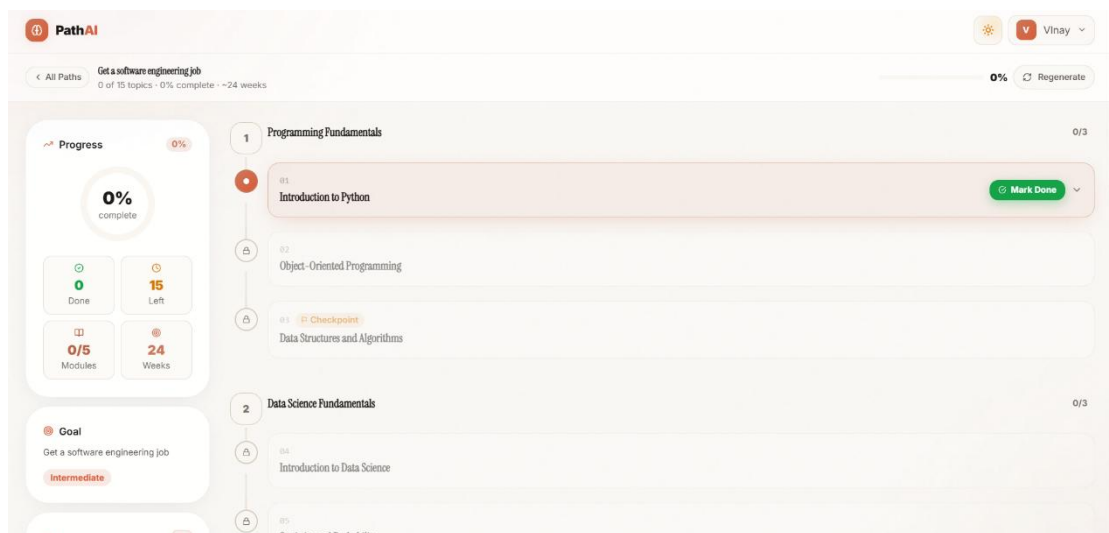


Figure 5.4: Roadmap Dashboard — Animated SVG Learning Path Timeline

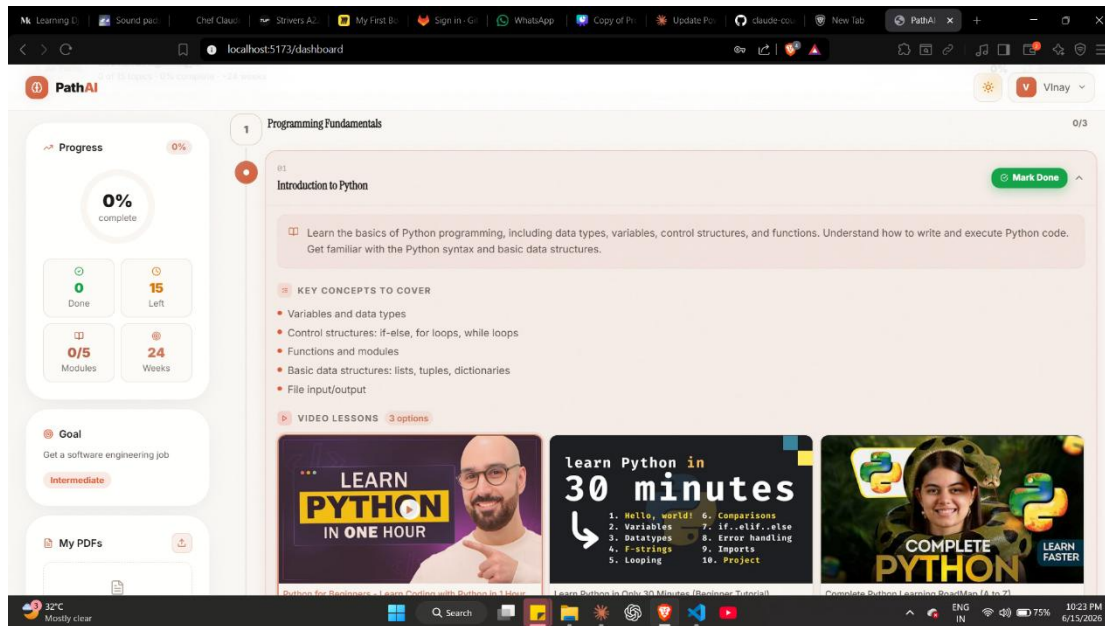


Figure 5.5: Expanded Topic Card — Description and Embedded YouTube Video

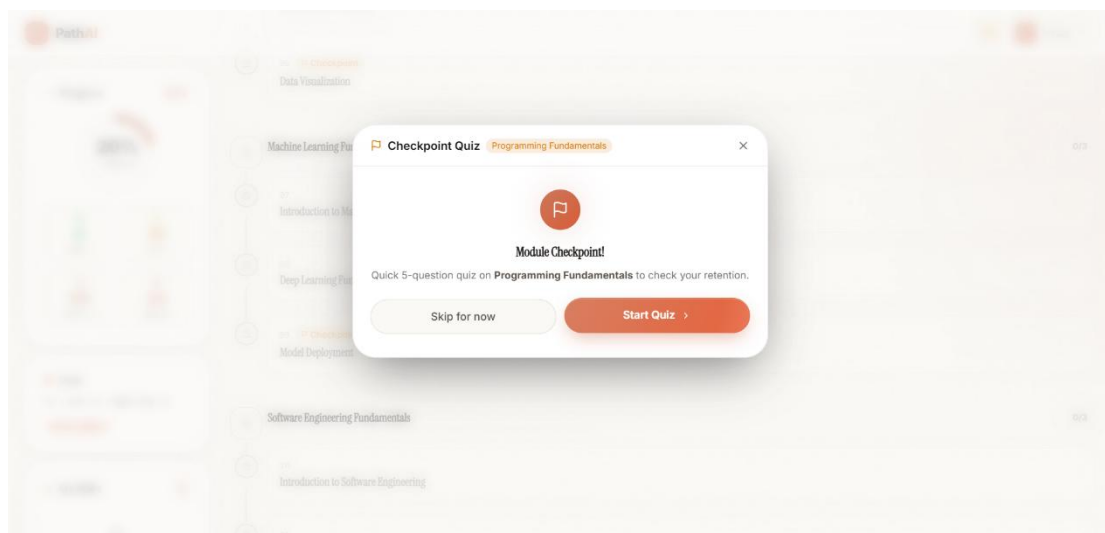


Figure 5.6: Checkpoint Quiz Modal — Module Mastery Assessment

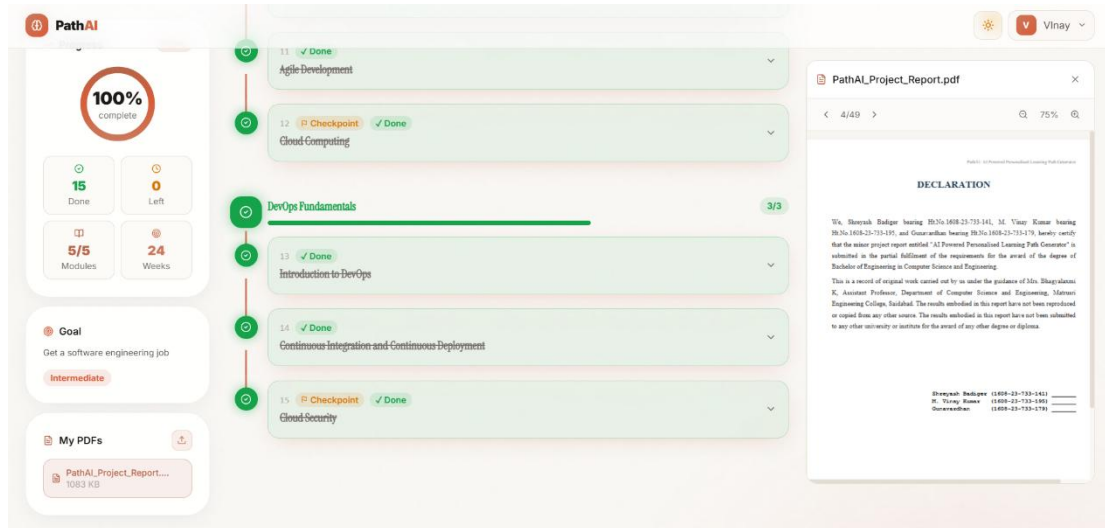


Figure 5.7: PDF Sidebar Panel — Custom Document Upload and Listing

5.3 Detailed Test Cases

The following test case tables document the inputs, expected behaviors, and actual outcomes recorded during system testing, demonstrating correctness across all core functional requirements.

Table 5.1: User Authentication Testing

Test ID	Description	Input Data	Expected Result	Actual Result	Status
AUTH_001	Valid student sign up	email='student@m.edu' , password='Password123'	Account created in Supabase Auth, session established.	Account created, session established.	PASS
AUTH_002	Valid student login	email='student@m.edu' , password='Password123'	User logs in, JWT token returned, redirected to /home.	Logged in successfully, token returned.	PASS

Test ID	Description	Input Data	Expected Result	Actual Result	Status
AUTH_003	Invalid credentials	email='student@m.edu', password='WrongPassword'	Login rejected, 'Invalid credentials' error displayed.	Error message displayed, login blocked.	PASS
AUTH_004	Empty field validation	email="", password=""	Form submission blocked, required field errors shown.	Fields flagged, submission blocked.	PASS

Table 5.2: Onboarding & Quiz Generation Testing

Test ID	Description	Input Data	Expected Result	Actual Result	Status
QUIZ_001	Submit onboarding form	field='Cloud Computing', level='beginner', goal='AWS Certification'	Data saved to onboarding_data, redirected to quiz.	Data stored correctly, quiz page loaded.	PASS
QUIZ_002	Generate diagnostic quiz	Onboarding data sent to POST /api/quiz/generate	Groq API generates 10 MCQ questions relevant to Cloud Computing.	10 questions returned and rendered.	PASS
QUIZ_003	Grade diagnostic quiz	7 correct answers submitted out of 10	Score = 70%, actual_level computed as 'intermediate'.	Score graded correctly, level set to intermediate.	PASS
QUIZ_004	Low score classification	2 correct answers	Score = 20%, actual_level	Correctly classified	PASS

Test ID	Description	Input Data	Expected Result	Actual Result	Status
		submitted out of 10	computed as 'beginner'.	as beginner level.	

Table 5.3: Roadmap & YouTube Integration Testing

Test ID	Description	Input Data	Expected Result	Actual Result	Status
PATH_001	Generate learning path	Profile + quiz score 70% sent to /api/learning-path/generate	Groq compiles intermediate-level JSON path (4-6 modules, 15-25 topics), written to DB.	Path JSON parsed, all path_topics records inserted.	PASS
PATH_002	Fetch YouTube resource	topic_title='AWS S3 Introduction'	Backend queries YouTube Data API, caches video metadata in path_topics.youtube_data.	Video metadata cached; video renders in dashboard player.	PASS
PATH_003	Track topic completion	PATCH /api/learning-path/topics/{id}/complete	is_completed=true, completed_at=now() written to path_topics.	Topic marked complete, progress bar increments.	PASS
PATH_004	Switch active path	PATCH /api/learning-path/{id}/activate	Previous path deactivated, selected path flagged is_active=true.	Path switch reflected correctly on dashboard reload.	PASS

Table 5.4: Checkpoint Evaluation & PDF Upload Testing

Test ID	Description	Input Data	Expected Result	Actual Result	Status
CHECK_001	Trigger checkpoint modal	Last topic of Module 1 marked complete (is_checkpoint=true)	Checkpoint Quiz modal appears, Module 2 remains locked.	Modal triggered correctly, next module locked.	PASS
CHECK_002	Submit checkpoint quiz	4 of 5 correct answers submitted	Score=4/5 written to checkpoint_results, Module 2 unlocked.	Correctly scored and module unlocked.	PASS
CHECK_003	Fail checkpoint quiz	1 of 5 correct answers submitted	Score=1/5 recorded, module remains locked, retry option shown.	Module locked, retry prompt displayed.	PASS
PDF_001	Upload reference PDF	File='AWS_S3_Whitepaper.pdf' (2 MB)	File uploaded to Supabase 'user-pdfs' bucket, public URL listed in PDF sidebar.	File uploaded, URL returned and rendered.	PASS

Chapter 6: Conclusion and Future Scope

6.1 Conclusion

The AI Powered Personalised Learning Path Generator (PathAI) platform successfully demonstrates the practical application of Generative AI in the domain of education technology. By replacing the traditional static curriculum model with a dynamic, diagnostic-quiz-driven roadmap generator, PathAI directly addresses cognitive overload and creates a highly structured, personalized self-learning environment for college students.

The complete integration of FastAPI, React.js, Supabase PostgreSQL, and the Groq Cloud API enables near-instant LLM inference and robust, persistent progress tracking. Features including real-time YouTube Data API v3 video embedding and custom PDF textbook uploads allow students to consolidate multiple learning media formats into a single unified dashboard. Modular checkpoint quizzes generated by the same LLM validate concept retention at each stage of the roadmap, creating a closed-loop, outcome-driven educational system.

The testing outcomes documented in Chapter 5 confirm that all core functional requirements are met correctly, establishing PathAI as a stable, flexible, and educationally meaningful platform for personalized self-directed learning.

6.2 Future Scope

While PathAI provides a complete diagnostic learning timeline, several enhancement areas have been identified for future development:

- **Real-Time Mentor Collaboration:** A dedicated teacher or mentor role could be introduced, allowing instructors to log in, inspect individual students' roadmaps, modify module content, add annotations, and submit feedback reviews directly within the platform.
- **Advanced AI Doubt Resolution Assistant:** A persistent, context-aware AI chatbot sidebar powered by the Groq API could be integrated, enabling students to ask questions about any active topic and receive instant, context-specific answers without leaving the dashboard.

- **Syllabus PDF Parsing:** Using Optical Character Recognition (OCR) and Large Language Model text analysis on uploaded PDF syllabi, the system could automatically generate tailored learning paths directly derived from official course outlines.
- **Mobile Application:** Building native Android and iOS companion applications using React Native or Flutter would allow students to access their roadmaps, track progress, and view resources offline — supporting continuous learning outside of a browser environment.
- **Adaptive Path Re-generation:** Based on accumulated checkpoint quiz performance data, the system could automatically re-calibrate the difficulty and ordering of remaining topics mid-roadmap, creating a truly adaptive and self-correcting curriculum.

References/Bibliography

- [1] T. FastAPI Contributors, "FastAPI: A Modern, Fast Web Framework for Building APIs with Python," FastAPI Documentation, Sebastián Ramírez, 2024. [Online]. Available: <https://fastapi.tiangolo.com/>. [Accessed: Jun. 2026].
- [2] Meta Open Source, "React: A JavaScript Library for Building User Interfaces," React Documentation, Meta Platforms Inc., 2024. [Online]. Available: <https://react.dev/>. [Accessed: Jun. 2026].
- [3] Groq Inc., "Groq Cloud API Reference — Llama 3.3 70B Versatile Model," Groq Developer Documentation, 2025. [Online]. Available: <https://console.groq.com/docs/>. [Accessed: Jun. 2026].
- [4] Supabase Inc., "Supabase Documentation — Open Source Firebase Alternative with PostgreSQL," Supabase Docs, 2024. [Online]. Available: <https://supabase.com/docs/>. [Accessed: Jun. 2026].
- [5] Google LLC, "YouTube Data API v3 Reference Guide," Google Cloud Developer Documentation, 2024. [Online]. Available: <https://developers.google.com/youtube/v3/>. [Accessed: Jun. 2026].
- [6] Meta AI Research, "Llama 3: Open Foundation and Fine-Tuned Chat Models," arXiv preprint arXiv:2407.21783, Jul. 2024. [Online]. Available: <https://arxiv.org/abs/2407.21783>. [Accessed: Jun. 2026].
- [7] S. Fryer and J. Carpenter, "Emerging Technologies and the Student Self: The Case of Artificial Intelligence and Personalized Learning," *Teaching in Higher Education*, vol. 25, no. 4, pp. 456-471, 2020.
- [8] C. Romero and S. Ventura, "Educational Data Mining and Learning Analytics: An Updated Survey," *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, vol. 10, no. 3, e1355, May 2020. doi: 10.1002/widm.1355.
- [9] P. Franca and A. Kaviani, "Adaptive Learning Systems: A Review of AI-Driven Curriculum Personalization Techniques," *IEEE Transactions on Learning Technologies*, vol. 15, no. 2, pp. 178-191, Apr. 2022. doi: 10.1109/TLT.2022.3152847.

[10] Framer Motion Contributors, "Framer Motion: A Production-Ready Motion Library for React," GitHub Repository, 2024. [Online]. Available: <https://www.framer.com/motion/>. [Accessed: Jun. 2026].